

1. Introdução

1.1. Motivação

O crescente interesse por ambientes inteligentes impulsiona o desenvolvimento de sistemas capazes de perceber o contexto ao redor do usuário e agir de forma autônoma, reduzindo a necessidade de interação explícita. Nesse cenário, a automação residencial e predial surge como uma das aplicações mais concretas dos conceitos de Computação Pervasiva e Ubíqua, integrando sensores, atuadores e comunicação sem fio em soluções invisíveis do ponto de vista do usuário.

Um dos desperdícios mais comuns em ambientes domésticos e de trabalho é o consumo desnecessário de energia elétrica com iluminação – luzes que permanecem acesas em ambientes vazios ou já suficientemente iluminados pela luz natural. Sistemas capazes de detectar presença e avaliar as condições de luminosidade do ambiente podem atuar de forma autônoma para mitigar este problema, contribuindo tanto para a eficiência energética quanto para o conforto do usuário.

1.2. Objetivos

O objetivo geral deste projeto é desenvolver um sistema embarcado distribuído de iluminação inteligente, capaz de detectar a presença de um usuário em uma área monitorada e controlar automaticamente uma carga de iluminação com base em múltiplas variáveis de contexto.

São objetivos específicos do projeto:

- Detectar presença por meio de sensor ultrassônico HC-SR04, com base em limiar de distância configurável;
- Incorporar leitura de luminosidade ambiente via sensor LDR para compor a decisão de acionamento;
- Estabelecer comunicação sem fio entre o dispositivo embarcado e um broker MQTT público via Wi-Fi, publicando eventos de forma assíncrona e orientada a transições de estado;
- Registrar e persistir o histórico de eventos de presença em banco de dados não relacional, por meio de um serviço *backend* desenvolvido em Python atuando como subscriber MQTT;
- Permitir ao usuário consultar graficamente o histórico de eventos e intervir manualmente no estado do sistema por meio de uma interface *frontend*, aproveitando a natureza bidirecional do protocolo MQTT para envio de comandos remotos ao ESP32.

2. Revisão bibliográfica

A Computação Pervasiva, conceito introduzido por Weiser (1991), propõe que a computação se integre ao ambiente de forma invisível, com dispositivos inteligentes colaborando silenciosamente para auxiliar o usuário em suas tarefas cotidianas. Nessa visão, o ambiente torna-se capaz de perceber contexto – localização, presença, temperatura, luminosidade – e reagir de forma adequada sem exigir interação explícita.

A detecção de presença é um dos elementos fundamentais em sistemas de automação ambiental. Sensores ultrassônicos, como o HC-SR04, são amplamente empregados nesse tipo de aplicação por seu baixo custo, facilidade de integração e capacidade de operar de forma confiável em ambientes internos. O princípio de funcionamento baseia-se na emissão de pulsos ultrassônicos e na medição do tempo de retorno do eco, permitindo calcular a distância até o objeto mais próximo com precisão da ordem de milímetros.

A fusão de múltiplos sensores para composição de contexto é uma prática consolidada em sistemas pervasivos. A combinação de dados de presença com leituras de luminosidade ambiente, obtidas por sensores LDR (Light Dependent Resistor), permite decisões mais refinadas

sobre o acionamento de iluminação artificial, evitando acionamentos desnecessários quando a luz natural já é suficiente.

Para a comunicação sem fio em sistemas IoT com múltiplos nós heterogêneos, o protocolo MQTT (Message Queuing Telemetry Transport) destaca-se por seu modelo publish/subscribe leve e eficiente. Nesse modelo, os dispositivos publicam mensagens em tópicos gerenciados por um broker centralizado, ao qual outros nós – independentemente de sua natureza, seja um microcontrolador, um serviço backend ou uma interface web – podem se inscrever para receber eventos de forma assíncrona. Essa arquitetura desacopla produtores e consumidores de dados, tornando o protocolo especialmente adequado a cenários pervasivos onde dispositivos embarcados de baixo poder computacional, como o ESP32, precisam se comunicar com sistemas mais complexos sem overhead de conexão direta. Para a persistência dos eventos coletados, bancos de dados não relacionais orientados a documentos, como o MongoDB, são amplamente adotados em arquiteturas IoT por sua flexibilidade de esquema e adequação ao armazenamento de séries temporais de eventos. Na camada de apresentação, frameworks reativos como o React permitem a construção de interfaces que refletem o estado do sistema em tempo real, consumindo dados do backend e, aproveitando a bidirecionalidade do MQTT, atuando também como publishers de comandos de controle remoto.

3. Materiais e método

3.1. Materiais

Os componentes a seguir serão utilizados ao longo dos três protótipos:

- 1× microcontrolador ESP32 (WROOM-32)
- 1× sensor ultrassônico HC-SR04
- 1× sensor LDR com resistor de 10kΩ (divisor de tensão)
- 1× LED externo com resistor de 220Ω
- Resistores para divisor de tensão do pino ECHO do HC-SR04 (adequação de 5V para 3,3V)
- Protoboard e cabos jumper
- Cabo USB para programação e alimentação
- Ambiente de desenvolvimento: IDE Thonny com MicroPython
- Dispositivo para execução de *backend*, como tablet ou computador

3.2. Método

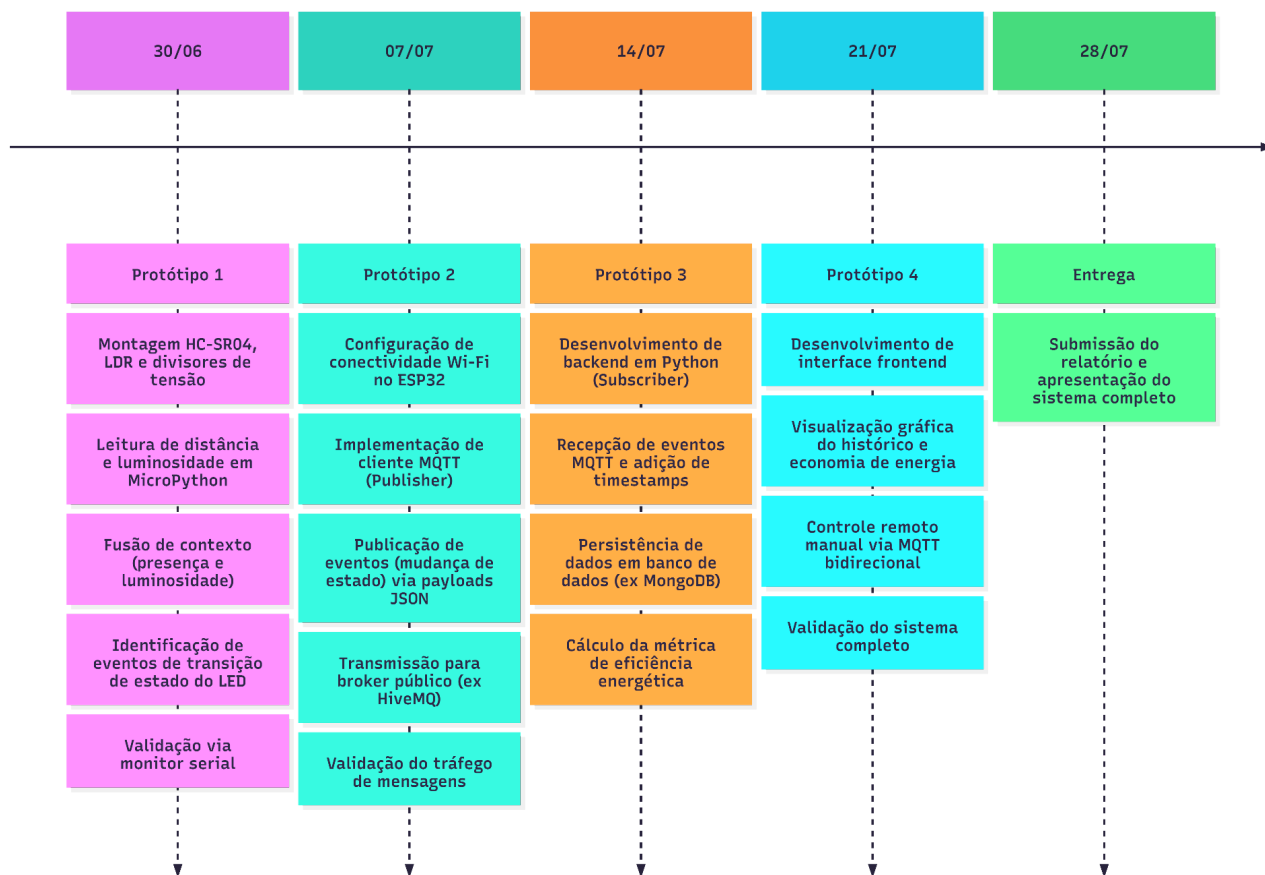
O desenvolvimento é organizado em três protótipos incrementais, cada qual funcional de forma independente e servindo de base para o seguinte:

- **Protótipo 1 - Detecção, fusão de contexto e controle local:** envolve o uso de um microcontrolador ESP32 integrado ao sensor ultrassônico HC-SR04 e ao sensor LDR. O sistema embarcado, desenvolvido em MicroPython utilizando a IDE Thonny, realiza a fusão de contexto em tempo real. A lógica de acionamento avalia duas condições simultâneas: a detecção de presença em um limiar de distância configurável e a luminosidade ambiente abaixo de um limiar predefinido. O LED externo é acionado estritamente quando ambas as condições são satisfeitas. A programação deve focar na identificação de eventos de transição de estado (momento exato em que a luz acende ou apaga), preparando o terreno para a comunicação assíncrona, com as leituras sendo validadas inicialmente via monitor serial.

- **Protótipo 2 - Conectividade Wi-Fi e publicação de eventos via protocolo MQTT:** mantendo a estrutura de sensoriamento e fusão de contexto do Protótipo 1, o ESP32 passa a atuar como um nó *publisher* em uma rede Wi-Fi. A comunicação sem fio é estabelecida utilizando o protocolo MQTT, enviando mensagens formatadas (payloads JSON) para um *broker* público (ex.: HiveMQ). A transmissão de dados é estritamente orientada a eventos: o microcontrolador publica o estado atual do sistema apenas quando ocorre uma mudança no estado da iluminação, garantindo eficiência de tráfego na rede e eliminando a dependência de varreduras contínuas (*polling*).
- **Protótipo 3 - Gateway computacional e persistência de dados:** o escopo se expande para fora do hardware embarcado, introduzindo um sistema computacional (um *backend* desenvolvido em Python) que atua como *subscriber* no tópico MQTT correspondente. Este serviço escuta continuamente os eventos publicados pelo ESP32, anexa *timestamps* precisos a cada transição de estado e persiste essas informações em um banco de dados não relacional (como MongoDB). Além do registro de eventos, o *backend* implementa a regra de negócio para calcular a métrica de eficiência energética, quantificando o tempo em que a carga de iluminação permaneceu desligada de forma autônoma durante o dia, em comparação a um cenário sem automação.
- **Protótipo 4 - Interface de visualização e atuação remota bidirecional:** o sistema completo é finalizado com a construção de uma interface de usuário (um *frontend* construído em React) que consome os dados processados pelo Protótipo 3. A interface permite ao usuário consultar de forma gráfica o histórico de eventos e o cálculo de economia de energia. Aproveitando a natureza bidirecional do protocolo MQTT, a interface também atua como *publisher* em um tópico de controle. O ESP32, agora configurado simultaneamente como *subscriber* deste tópico de controle, pode receber comandos remotos para forçar o estado do LED manualmente, sobrepondo a lógica autônoma baseada em sensores quando houver intervenção direta do usuário.

3.3. Plano de atividades

Cronograma de Implementação



4. Considerações finais

O projeto proposto aborda de forma direta os princípios de Computação Pervasiva e Ubíqua ao integrar sensoriamento de contexto, tomada de decisão autônoma e comunicação sem fio em um sistema de iluminação inteligente. A escolha de componentes e protocolos já trabalhados em laboratório garante viabilidade de execução dentro do tempo disponível, ao mesmo tempo em que a progressão entre protótipos permite validar cada camada do sistema de forma independente.

A divisão em quatro protótipos incrementais assegura que, mesmo diante de eventuais dificuldades técnicas, o projeto entregue ao final seja funcional e demonstre ao menos os conceitos centrais propostos. O escopo do sistema envolve o projeto de uma arquitetura IoT completa, incorporando protocolos adicionais e conceitos de eficiência energética relevantes para aplicações reais de automação ambiental.

5. Referencial bibliográfico

WEISER, M. The computer for the 21st century. **Scientific American**, v. 265, n. 3, p. 94–104, 1991.

Montagem protoboard.

Cabo MF: F Ground, M “-”

Cabo MF: F 3.3V, M “+”

Cabo MF: F D34, M B15

Cabo MF: F D4, M E4

Cabo MF: F D5, M C35

Cabo MF: F D18, M A29

Resistor Vermelho Vermelho Marrom: D4, D8

LED: C8, C10

Cabo MM: A10, -10

LDR: +15, A15

Resistor Dourado Laranja Preto Marrom: C15, C19

Cabo MM: A19, -19

Cabo MM: A25, -25

Resistor Vermelho Preto Vermelho Dourado: C25, C29

Resistor Marrom, Preto Vermelho Dourado: B29, B34

Cabo MM: D33, -33

Cabo MM: D36, +36

Sensor de Som: E33 a E36